ABRA — full-codebase conformance review

2026-07-26 · 132 source files · 23,909 lines of code · 7,554 lines of documentation

A pass over every file in the project against the standards the project set itself in `ARCHITECTURE.md` (S1–S13), plus the removal of dead files and stale claims.

The review is **executable**. `engine/conformance.js` encodes the standards as checks and prints the same report for anyone who runs it, so this document records what was found and fixed on one day rather than becoming the authority. Re-run it before believing any of the numbers below:

```
node engine/conformance.js
```

---

1. Why a tool and not a read-through

The standards were good and nothing enforced them. They were kept by whoever remembered, and over a single evening that produced, in the project's own code:

- a **hand-typed threshold** inside a file whose purpose is to argue against hand-typed thresholds
- a **hardcoded list** inside the tool written to catch hardcoded lists
- two models **quoted as evidence** without anyone checking what data they were built on
- a **Pokémon used in a demo that is not legal in the format**

Every one passed review by a human who had just written the rule it broke. A person reading 23,909 lines finds some of them; the same person reading again next month finds a different subset. So the standards are now machine-checked, and the check ships with the code.

---

2. What was found

standard	found	fixed	remaining
S12 everything linked, nothing hardcoded	13	**13**	0
S13 no hand-maintained state	18	5	13
S8 measured, never asserted	1	0	1
dead / unreachable files	19 → 1 real	3 files deleted	1
file-header convention	13	0	13

The check itself was wrong three times before it was trusted

This is the most important line in the report, because it is the failure mode the tool exists to prevent, reproduced by the tool.

1. **It called `engine/bring_priors.js` dead.** The hourly ingest workflow invokes it by name every hour. **Acting on that would have stopped data collection.** A file is also reached by workflows, shell scripts and documented commands — not only by `require`.

2. **It called** `build/build_engine_data.js` **dead.** It generates `data/engine-data.js`, which the entire site loads. Producing a live artifact is the strongest possible evidence a file is reached.
3. **It reported 13 hardcoded format ids when 3 were real.** The other ten were the format named in *prose*, explaining itself — documentation doing its job. The checker now strips comments before looking.

A checker that cries wolf is one people learn to scroll past, which is how the project reached this state. False positives were treated as more serious than gaps throughout.

---

3. S12 — the format id: 13 copies, now 1 source

`engine/champions_sim.js` declared `const FORMAT = 'gen9championsvgc2026regmb'` and eleven other files restated it: `analyze.js`, `chomp_ev.js`, `meta-ingest.js`, `validate_damage_sim.js`, `sim/champions-battle.js`, and more.

`data/regulations.json` already held it, and `durable-ingest.js` and `fetch_smogon_stats.js` already read it. Everything now does. **When Reg M-B rotates, every copy would have kept describing a metagame that no longer exists — and nothing would have noticed, because a stale format id produces plausible output rather than an error.**

The literal survives in one place per file as the fallback for a corrupt config, which is the single case where a sensible guess beats crashing a collection job.

One careless automated edit inserted a helper between a `try` and its `catch` in `sim/champions-battle.js`, breaking the file. Caught by `node --check` before commit; every edited file is now syntax-checked as a matter of course.

---

4. Dead files removed

Three files, all swept into the repository by the auto-commit watcher on 23 July — the unattended publisher `CLAUDE.md` warns about:

file	size	referenced by	verdict
`data/display-maps.json`	19 KB	**nothing**	deleted
`data/real-sets.json`	10 KB	**nothing**	deleted
`data/nontransitivity.json`	644 B	three documents, no code	deleted — see below

The stale claim that came with it

`docs/MODELS.md` stated as a live fact: *"the meta is rock-paper-scissors"*, citing `data/nontransitivity.json`.

That file was computed **2026-07-23, two days before the quality filter existed**, and nothing ever regenerated it. Its cycles were measured over a corpus that is 87% bots, forfeits and stubs.

Re-run on clean data the same evening, SLOWKING's equilibrium collapses to **100% on a single option**, with **zero** gap between mixing and simply picking the best, and the clean GURU matrix contains **0 decisive matchups**. The claim is withdrawn in `MODELS.md` with the evidence.

The file was deleted rather than kept, because a stale artifact on disk is how a retracted claim gets quoted again — which is exactly what happened to it.

The honest reading is *no usable input*, not *mixing does not help*: a Nash solution over a matrix of noise says nothing in either direction.

---

5. Still open — the burn-down

Catalogued, not fixed. `node engine/conformance.js` lists them precisely.

S13 — 13 generated artifacts do not say they are generated. `calibration.json`, `eval-report.json`, `jolteon-weights.json`, `meta-nash.json`, `value-net.json`, `trajectories.sample.json`, `pory-nn.json` and the browser bundles `live.js`, `pory.js`, `xatu.js`, `slowking.js`, `slowking-playstyle.js`, `kad-replays.js`. Each is produced by a known generator; each generator needs to stamp its output. `engine/stamp.js` defines the one block they should all carry.

S8 — one undeclared constant. `build/triggers.js` sets `Z_ALPHA = 1.959964` with no stated derivation. It is almost certainly the 95% normal quantile, which is fine — but the standard says a constant that decides must say where it came from, and this one does not.

Convention — 13 files without a proper header. `analyze.js`, `build_mega_dex.js`, `build_species_abilities.js`, `game-spec.js`, `illusion.js`, `ingest_ots.js`, `kadabra.js`, `archive-regulation.js`, two test files, and the three HTML pages. Every other file in this project opens with a paragraph explaining what failure it prevents, and that convention is the main reason the code is readable.

Dead — 1 candidate. `engine/chomp-predict.js` is named in two documents but nothing runs it. Needs a decision: wire it up or delete it.

---

6. Related findings from the same day

Recorded here because they are input-quality problems of the same family, and are covered in full in `CHANGELOG.md` 3.16.0 – 3.19.0.

- `engine/provenance.js` now audits every published artifact for staleness against the quality filter, against its own inputs, and against the clean-game count. It found **28 artifacts unsafe to quote**, including one quoted in conversation the same hour.
- `pory_nn.py` **had an opt-in filter.** `--clean` defaulted to *off*, so the lazy path trained on the raw archive — which is how `data/pory-nn.json` came to declare 61,274 games against a clean store of ~2,000. Four other models already had it the right way round; it now matches, and the audit fails the build if anyone reintroduces the shape.
- **The raw games are kept deliberately.** They are what the behavioural bot detector identifies bot *accounts* from, they yield 14 ability rules against 11 from clean-only, and they are needed to measure the scrape's own bias. The project's rule stands: *store raw, analyse on top*.

---

7. How to keep this true

```
node engine/conformance.js --strict     # standards; non-zero exit on any violation
node engine/provenance.js  --strict     # artifact freshness; non-zero if anything is unsafe
node engine/selftest.js                 # the silent-wrongness assertions
```

All three are cheap and none needs a Showdown checkout for the majority of their checks. The intention is that they run in CI beside the existing test suite, so a standard cannot be broken quietly — which is the only mechanism that has ever worked here.